

JAVA 8 :

➤ WHY JAVA 8 ? MAIN AGENDA BEHIND JAVA 8?

TO MAKE CONCISE CODE, JAVA BRINGS IN FUNCTIONAL PROGRAMING WHICH IS ENABLED BY LAMBDA EXPRESSIONS.

➤ WHAT ARE NEW FEATURES WHICH GOT INTRODUCED IN JAVA 8 ?

LAMBDA EXPRESSION, STREAM API, DEFAULT METHOD IN INTERFACE, STATIC METHOD IN INTERFACE, FUNCTIONAL INTERFACE, OPTIONAL CLASS , METHOD REFERENCE, DATE API.

➤ WHAT ARE THE MAIN ADVANTAGES USING JAVA 8?

LESS BOILER PLATE CODE, MORE READABLE AND REUSABLE CODE, MORE TESTABLE CODE, PARALLEL OPERATIONS.

➤ WHAT IS LAMBDA EXPRESSION?

➤ WHAT ARE FUNCTIONAL INTERFACES?

➤ HOW LAMBDA EXPRESSION AND FUNCTIONAL INTERFACES ARE RELATED?

FUNCTIONAL INTERFACES ARE USED TO PROVIDE REFERENCE TO LAMBDA EXPRESSION.

➤ CAN YOU CREATE YOUR OWN FUNCTIONAL INTERFACE?

➤ WHAT IS THE METHOD REFERENCE IN JAVA 8?

METHOD REFERENCE IS THE REPLACEMENT OF LAMBDA EXPRESSION, IT IS USED TO REFER METHOD OF FUNCTIONAL INTERFACE TO AN EXISTING METHOD , MAINLY IT IS USED FOR CODE REUSEABILITY ,

WHENEVER YOU HAVE EXISTING IMPLEMENTATION OF ABSTRACT METHOD OF OUR FUNCTIONAL INTERFACE THEN WE CAN GO FOR METHOD REFERENCE , IF NO SUCH METHOD IS AVAILABLE THEN GO FOR LAMBDA EXPRESSION.

➤ What are default methods?

DEFAULT METHOD IS A WAY FOR ADDING NEW METHODS TO INTERFACE WITHOUT AFFECTING THE IMPLEMENTING CLASSES IT PROVIDE THE BACKWARD COMPATIBILIT.

➤ IS IT NECESSARY TO OVERRIDE DEFAULT METHOD?

➤ IS DEFAULT KEYWORD ONE OF THE ACCESS MODIFIER?

DEFAULT IS NOT THE ACCESS MODIFIER LIKE PUBLIC , PRIVATE OR PROTECTED. FOR DEFAULT ACCESS MODIFIER WE DON'T USE ANY KEYWORD.

DEFAULT KEYWORD WAS ONLY USED IN CLASSES TILL 8 VERSION FOR SWITCH CASE ONLY BUT NEVER IN INTERFACE .

NOW IT USED FOR DEFAULT METHOD IN INTERFACE TO PROVIDE A DEFAULT IMPLEMENTATION FOR ALL IMPLEMENTING CLASSES TO USE.

- HOW DEFAULT METHODS IN INTERFACE WITH DIAMOND PROBLEM/ AMBIGUITY PROBLEM?

IF TWO IMPLEMENTED INTERFACES CONTAINS SAME DEFAULT METHOD THEN THAT'S THE DIAMOND PROBLEM IN SUCH SITUATION USER : INTERFACE NAME. SUPER.METHOD NAME();

- WHY STATIC METHODS WERE INTRODUCED IN JAVA 8?

YOU CAN CALL THOSE METHODS WITH JUST INTERFACE NAME . NO NEED TO CREATE CLASS AND THEN IT'S OBJECT BECAUSE INTERFACE NEVER CONTAINS : CONSTRUCTOR, STATIC BLOCKS SO NOTHING COSTLY IN TERMS OF MEMORY AND PERFORMANCE

YOU CAN'T OVERRIDE ALSO.

- WHAT ARE PREDICATE?

- WHEN TO USE PREDICATE?

- HOW MANY TYPE PARAMETER IT'S TAKE AND WHAT IS THE RETURN TYPE OF PREDICATE?

- WHAT IS THE ADVANTAGE OF PREDICATES ?

DECLARE ONCE AND USE THE MORE TIME YOU WANT

- WHAT IS PREDICATE JOINING?

COMBINE PREDICATE IN SERIAL PREDICATE.

THERE ARE THREE WAY TO JOIN: AND, OR, NEGATE

- WHAT ARE FUNCTIONS?

- WHAT ARE THE DIFFERENCE BETWEEN PREDICATE AND FUNCTION?

- WHAT IS FUNCTIONAL CHAINING?

COMBINE MULTIPLE FUNCTION TOGETHER WITH AND THEN () .

THERE ARE TWO WAYS TO COMBINE FUNCTIONS:

F1.AND THEN(F2).APPLY(INPUT)=FIRST F1 THEN F2,

F1.COMPOSE(F2).APPLY(INPUT)= FIRST F2 THEN F1

- WHAT IS CONSUMER FUNCTIONAL INTERFACE?

- WHAT IS CONSUMER CHAINING ?

WE COMBINE MULTIPLE CONSUMER TOGETHER WITH AND THEN.

THERE IS ONLY ONE WAY TO COMBINE CONSUMER:

C1.ANDTHEN(C2) .APPLY(INPUT) ; FIRST C1 THEN C2 IT DOESN'T HAVE COMPOSE() METHOD LIKE FUNCTIONAL CHAINING.

- WHAT IS SUPPLIER FUNTIONAL INTERFACE?
- USE OF BICONSUMER, BIFUNCTION, BIPREDICATE AND WHY NO BISUPPLIER?

Q) Use of Biconsumer BiFunction, BiPredicate and Why no BiSupplier?

- Till now we had :
 - Predicate<T> → test() → return boolean
 - Function<T, R> → apply() → returns anything
 - Consumer<T> → accept() → returns nothing
 - Supplier<R> → get() → returns anything
- What if we need 2 arguments for operation?
- Then we need Bi XYZ Functional Interfaces.
- There is no input in supplier so no 1 or 2 Input arguments needed. Hence no BiSupplier.

- IF WE WANT TO OPERATE ON 3 ARGUMENTS THEN TRIPREDICATE IS THERE OR TRI FUNCTION, TRISUPPLIER?

NO

- WHAT ARE STREAMS?

Q) What are streams?

- If we want to process bulk objects of collection then go for streams concept.
- Way to operate on collection in java 8 is Stream.
- Its a special iterator class that allows processing collections of objects in a functional manner.
- Eg : fetch all objects from collection of list whose value is greater than 15

(1) What is stream ?

Stream: stream is a feature in java 8 through which process collections of data.

(2) How many types of stream ?

There are two types:

(a) Sequential stream

(b) Parallel stream

(3) Tell some methods to create stream from the collection of data?

Ex-

```
public static void main(String[] args) {  
  
    // ways to create streams  
  
    // using collections  
    List<Integer> list = Arrays.asList(1,2,3,4,5);  
    list.stream();  
  
    // from set stream created  
    Set<Integer> set = new HashSet<>();  
    Stream<Integer> stream = set.stream();  
  
    // from map stream created  
    Map<String,Integer> map = new HashMap<>();  
    Stream<Map.Entry<String, Integer>> stream1 = map.entrySet().stream();  
    Stream<Map.Entry<String, Integer>> entryStream =  
    map.entrySet().parallelStream();  
  
    // from array  
    String [] arr = {"s","n","m"};
```

```

    Arrays.stream(arr);

    // stream.of()
    Stream<String> sahil = Stream.of("sahil", "alam", "java");

    // infinite stream
    // one way
    Stream<Integer> generate = Stream.generate(() -> 1);

    // another way
    Stream.iterate(1, x-> x+1).limit(100);
}

```

- WHAT ARE DIFFERENCE BETWEEN STREAM JAVA 1.8 AND JAVA.IO.STREAM?
- DIFFERENCE BETWEEN STREAM JAVA 1.8 AND COLLECTION?

Q) Difference between streams (java 1.8) and collection?

- To represent group of collection as single entity then we should use collection concept.
- If we want to perform operation on bulk objects in collection then we should go for Streams.

- STEPS TO CREATE STREAM AND PROCESS STREAM?

Top Java 8 Streams API Interview Questions and Answers | Parallel Stream | Code Decode

Q) Steps to create and process stream

- We can get stream object by :
 - Stream s = collectionObject.stream();
 - Once we get stream object we can process the object of collection.
 - Processing of stream consists of 2 steps/ stages
 - Configuration of stream
 - Processing that configuration
 - Configuration can be done by
 - Map
 - Filter

7:58 / 19:53

Q) Steps to create and process stream

```

graph TD
    Streams[Streams] --> Config[Configuration of Stream]
    Streams --> Process[Processing that configuration]
    Config --> Collect[Collect]
    Config --> Count[Count]
    Config --> Sorted[Sorted]
    Config --> MinMax[Min Max]
    Config --> ForEach[For Each]
    Config --> toArray[toArray]
    Config --> of[of()]
    Process --> Filter[Filter]
    Process --> Map[Map]
    Process --> flatMap[flatMap]
    Process --> flatMapSeq[flatMapSequential]
    Process --> parallel[parallel]
    Process --> parallelOrdered[parallelOrdered]
    Process --> sequential[sequential]
    Process --> toArrayOrdered[toArrayOrdered]
  
```

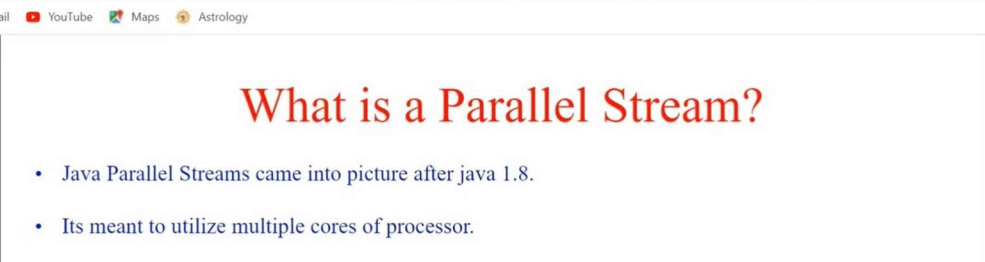
Slide 301

- WHAT IS DIFFERENCE BETWEEN FILTER AND MAP?
- HOW TO PROCESS ELEMENTS USING OF()?

Q) How to Process elements using of()?

- Stream concept is not applicable just for the collections it's also applicable for **"ANY GROUP OF VALUE"**.
- Even for arrays you can use stream.
- Stream . Of() this method can take any group of values and convert them to stream so that multiple stream operations can be applied to it.
- Demo

➤ WHAT IS PARELLEL STREAM?



java_27_2_21.pptx - Google Slide

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.gc4495bb158_2_75

What is a Parallel Stream?

- Java Parallel Streams came into picture after java 1.8.
- Its meant to utilize multiple cores of processor.
- Till Now our java code has 1 stream of processing where it executes sequentially.
- But when you use parallel streams, we divide code into multiple streams that executes parallely , on separate cores and final result is the outcome of individual cores outcomes combined.

- ## Parallel Stream?

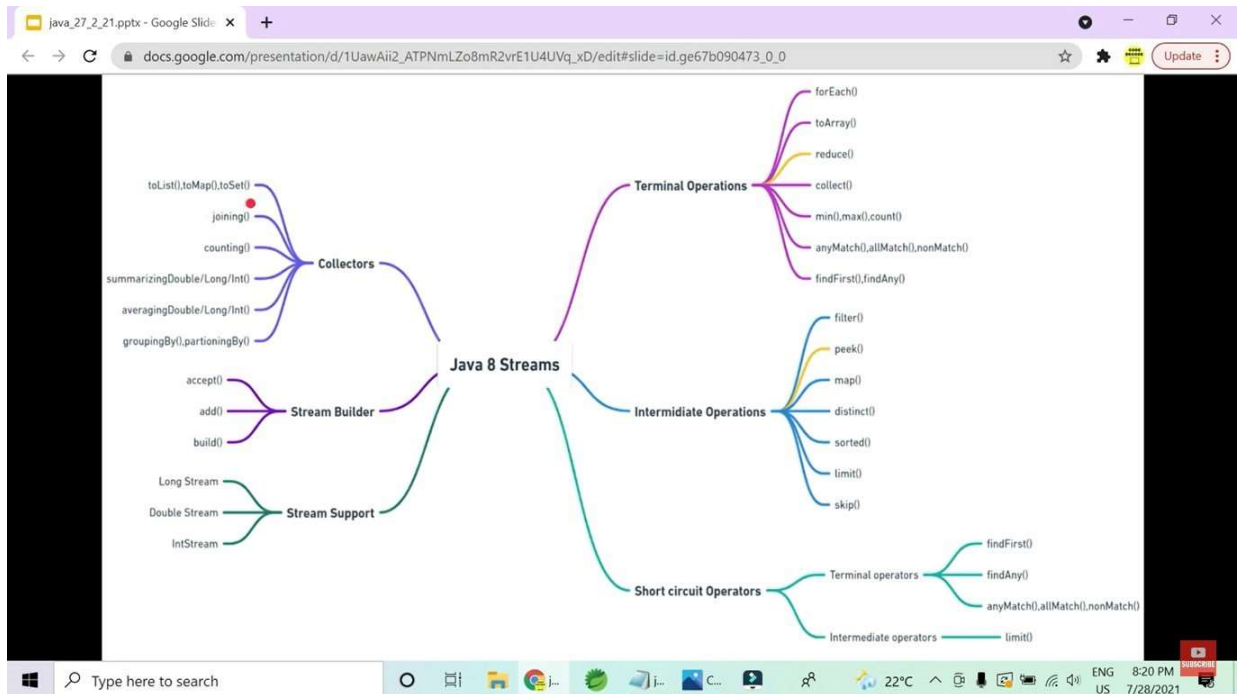
Tasks	Core	Task 1	Task 2	Task 3	Task 4
T1	CORE 1		T2		
T2	CORE 2				T4
T3	CORE 3	T1			
T4	CORE 4			T3	

 - The output of this Parallel stream is T2 , T4, T1 , T3 → not in sequential order.
 - Order of execution is not under control.
 - Hence its advisable to use parallel stream only when order of execution of threads does not matter and state of one element does not affect another

Tasks	Core	Task 1	Task 2	Task 3	Task 4
T1	CORE 1		T2		
T2	CORE 2				T4
T3	CORE 3	T1			
T4	CORE 4			T3	

- The output of this Parallel stream is T2 , T4, T1 , T3 → not in sequential order.
- Order of execution is not under control.
- Hence its advisable to use parallel stream only when order of execution of threads does not matter and state of one element does not affect another

➤ ARCHITECTURE OF JAVA 8 STREAM ?



➤ WHAT IS AN INTERMEDIATE OPERATION?

➤ WHAT IS DIFFERENCE BETWEEN INTERMEDIATE OPERATION AND TERMINAL OPERATION?

Terminal VS Intermediate Operations	
Intermediate Operations	Terminal Operations
They return stream.	They return non-stream values.
They can be chained together to form a pipeline of operations.	They can't be chained together.
Pipeline of operations may contain any number of intermediate operations.	Pipeline of operations can have maximum one terminal operation, that too at the end.
Intermediate operations are lazily loaded.	Terminal operations are eagerly loaded.
They don't produce end result.	They produce end result.

Intermediate operations:

Ex-

```
package org.example.JAVA8_5.STREAM.INTERMEDIATEOPERATIONS;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Stream;

public class InterMediateOp_01 {
    public static void main(String[] args) {

        // filter
        List<String> list = Arrays.asList("sahil", "sohail", "rahul", "afan",
"rani");
        list.stream().filter(name -> name.startsWith("s"));

        // map
        list.stream().map(name -> name.toLowerCase());

        // sorted
        list.stream().sorted();
        list.stream().sorted((name1, name2) -> name1.length() - name2.length());

        // distinct
        list.stream().distinct();

        // limit
        Stream.iterate(1, x -> x + 1).limit(20);

        // skip
        Stream.iterate(1, x -> x + 1).skip(2).limit(20);

        // peek
        Stream.iterate(1, x -
> x + 1).skip(2).limit(20).peek(System.out::println).count();

        // FLATMAP
        // flatmap: handle stream of collections, lists, or arrays where each
element is itself a collection
        // flatten nested structures (e.g- lists within lists) so that they can
be processed as a single sequence of elements
        // transform and flatten at same time
        // ex - 1
        List<List<String>> lists = Arrays.asList(
            Arrays.asList("Apple", "Banana"),
            Arrays.asList("Orange", "Kiwi"),
            Arrays.asList("pear", "grape")
        );

        lists.stream().flatMap(li -> li.stream()).map(s -
> s.toUpperCase()).forEach(System.out::println);

        // ex - 2
        List<String> sent = Arrays.asList(
            "Hello World",
            "Java Stream are powerful",
            "flatmap is powerful"
        );

        sent.stream().flatMap(s -> Arrays.stream(s.split(" ")))
```

```

""))).map(String::toUpperCase).forEach(System.out::println);

    }
}

```

terminal operations:

ex-

```

package org.example.JAVA8_5.STREAM.TERMINALOPERATIONS;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class TerminalOps_01 {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1,2,3,4);

        // collect
        List<Integer> collect =
list.stream().skip(2).collect(Collectors.toList());

        // forEach
        list.stream().forEach(s-> System.out.println(s));

        // reduce: combine elements to produce a single result
        Optional<Integer> reduce = list.stream().reduce((x, y) -> x + y);

        // count
        long count = list.stream().count();

        // anyMatch
        boolean b = list.stream().anyMatch(x -> x % 2 == 0);

        // allMatch
        boolean b1 = list.stream().allMatch(x -> x > 0);

        // noneMatch
        boolean b2 = list.stream().noneMatch(x -> x < 0);

        // findFirst
        Optional<Integer> first = list.stream().findFirst();

        // findAny
        Optional<Integer> any = list.stream().findAny();

        // toArray

        // min / max
        Stream.of(22,14,78).max(Comparator.naturalOrder());

        // Example : find name whose length > 3
        List<String> name = Arrays.asList("Anna","Bob","Charlie","David");
        name.stream().filter(s->s.length()>3).forEach(s->

```

```

System.out.println(s));

    // Example: Squaring and sorting numbers
    List<Integer> numbers = Arrays.asList(5,2,9,1,6);
    numbers.stream().map(n->n*n).sorted().forEach(s->
System.out.println(s));

    // Example: summing value
    List<Integer> num = Arrays.asList(5,2,9,1,6);
    Integer integer = num.stream().reduce((x, y) -> x + y).get();
    System.out.println(integer);

    // Example: counting occurrence of characters like 'l'
    String sentence = "Hello World";
    long count1 = sentence.chars().filter(s -> s == 'l').count();
    System.out.println(count1);
}
}

```

➤ WHAT IS PEEK()?

The image shows a Google Slides presentation with two slides. The top slide is titled "What is Peek" in red text and contains a bulleted list of four points about the `Stream peek()` method. The bottom slide is titled "Use of Peek" in red text and contains two paragraphs explaining the purpose and usage of the `peek()` method. The presentation is viewed in a web browser, and the Windows taskbar is visible at the bottom of the screen.

What is Peek

- Stream `peek()` method is an intermediate operation.
- It takes a consumer object as an input
- It returns a `Stream` consisting of the elements of current stream.
- It additionally perform the provided action on each element as elements.

Use of Peek

`peek()` exists mainly to support debugging, where we want to see the elements as they flow past a certain point in a pipeline.

Its is similar to `Map`, but it takes consumer object and perform some action on object and return nothing, But `map` takes a function argument hence apply operation on each element and return the stream having modified elements.

➤ WHAT IS REDUCE()?

The screenshot shows a Google Slides presentation titled "What is Reduce". The slide content includes a title and a bulleted list. The browser's address bar shows the Google Docs URL. The Windows taskbar at the bottom displays the search bar and various application icons.

What is Reduce

- the `Stream.reduce()` combine elements of a stream and produces a single value.
- reduce operation applies a binary operator to each element in the stream where the first argument to the operator is the return value of the previous application and second argument is the current stream element

➤ JAVA 8 STREAM, SHORT CIRCUITING OPERATIONS?

The screenshot shows a Google Slides presentation titled "Java 8 Streams - Short circuiting operations". The slide content includes a title and a bulleted list. The browser's address bar shows the Google Docs URL. The Windows taskbar at the bottom displays the search bar and various application icons.

Java 8 Streams - Short circuiting operations

- Java 8 short-circuiting operations are just like boolean short-circuit evaluations in Java.
- In boolean short-circuiting logic, for example `firstBoolean && secondBoolean`, if `firstBoolean` is false then the remaining part of the expression is ignored (the operation is short-circuited) because the remaining evaluation will be redundant. Similarly in `firstBoolean || secondBoolean`, if `firstBoolean` is true the remaining part is short-circuited.
- Java 8 Stream short-circuit operations are not limited to boolean types. There are pre defined short-circuiting operations.
- Java 8 stream intermediate and terminal operations both can be short circuiting.
- Intermediate - `limit()`
- Terminal - `findFirst()`, `findAny()`, `anyMatch()`, `allMatch()`, `noneMatch()`

➤ INTERMEDIATE SHORT CIRCUITING METHOD?

The image shows a Google Slides presentation titled "Intermediate short-circuiting methods" and "Rules - limit()". The presentation is viewed in a web browser with a Windows taskbar at the bottom.

Intermediate short-circuiting methods

- This method takes one (long N) as an argument and returns a stream of size no more than N
- **Stream<T> limit(long N)** - Where N is the number of elements the stream should be limited to and this function returns new stream as output.
- Use Java 8 Stream.limit() method to retrieve only the first n objects and setting the maximum size. And it ignores the remaining values after size n. Stream.limit(long maxSize) returns a Stream of objects.
- As soon as limit() reaches the maximum number of items, it doesn't consume any more items and simply returns the resulting stream. Hence, we say that limit() is a short-circuiting operation.
- limit() returns the first n elements in the encounter order and not just any n elements

Rules - limit()

- This is an intermediate operation.
- This is a good choice when working with infinite streams or infinite values.
- limit() method is invoked after calling the terminal operation such as count() or collect() methods.
- This returns a stream of elements with the size or max limit given.
- This works well in sequential streams. Not suggested using in the parallel streams such as larger or higher in size.
- maxSize can not be negative. If negative then it will throw IllegalArgumentException.

➤ TERMINAL SHORT CIRCUITING METHODS?

Terminal short-circuiting methods

- `Optional<T> findFirst()`
- Returns the very first element (wrapped in Optional object) of this stream and before traversing the other.
- The terminal operation terminated on finding the first element, hence short-circuited.
- Please note that, If source is not initially ordered then we cannot predict findFirst will still return '3'.

Terminal short-circuiting methods

- `Optional<T> findAny()`
- Returns an Optional instance which wraps any and only one element of the stream.
- The behavior of this operation is explicitly nondeterministic; it is free to select any element in the stream. This is to allow for maximal performance in parallel operations; the cost is that multiple invocations on the same source may not return the same result. (If a stable result is desired, use findFirst() instead.
- For a sequential stream there won't be any difference between 'findFirst' and 'findAny'. But for a parallel stream findAny will return 'any' element rather than waiting for the 'first' element.

The image shows a Google Slides presentation titled "Terminal short-circuiting methods". The slide content is as follows:

Terminal short-circuiting methods

- boolean anyMatch(Predicate<? super T> predicate)
- Tests whether any elements of this stream match the provided predicate. This terminal method will return as soon as it finds the match and will not transverse all the remaining elements to apply the predicate.

Terminal short-circuiting methods

- boolean allMatch(Predicate<? super T> predicate)
- Tests whether all elements match the provided predicate. It may return early with false result when any element doesn't match first.

The screenshot also shows the Windows taskbar at the bottom with various application icons and system status indicators like temperature (21°C) and time (6:55 PM 2/3/2022).

java_27_2_21.pptx - Google Slide x +

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mP... To exit full screen, press Esc

Terminal short-circuiting methods

- `boolean noneMatch(Predicate<? super T> predicate)`
- Tests whether no elements of this stream match the provided predicate. It may return early with false result when any element matches the provided predicate first.

Type here to search

De... Wo... jav... Pos... Mic... Au... 21°C 7:00 PM 2/3/2022

➤ DIFFERENCE BETWEEN LIMIT() AND SKIP() METHOD?

Java 8 Programming Interview Q x +

docs.google.com/presentation/d/1weo7RmcOjQFh9hMRjL0n1po8ZEgcmP86iY2kXvji0E/edit#slide=id.ge476d7780c_0_608

What's the difference between limit() and skip() ? explain using example ?

Limit : The limit(n) method is an intermediate operation that returns a stream not longer than the requested size. As before, the n parameter can't be negative.

Skip : The skip(n) method is another intermediate operation that discards the first n elements of a stream. The n parameter can't be negative, and if it's higher than the size of the stream, skip() returns an empty stream.

Type here to search

23°C 5:34 PM 7/30/2021

➤ WHAT IS DIFFERENCE BETWEEN MAP AND FLATMAP?

java_27_2_21.pptx - Google Slide x +

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g103a5b83786_0_0

How Map works

- The Stream.map() function performs map functional operation i.e. it takes a Stream and transforms it to another new Stream.
- It applies a function on each element of Stream and stores return value into new Stream.
- The map operation takes a Function, which is called for each value in the input stream and produces one result value, which is sent to the output stream.

Type here to search

Wor... *ne... Won... java... Micr... Aud... 25°C 5:39 PM ENG IN 11/27/2021

java_27_2_21.pptx - Google Slide x +

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g103a5b83786_0_0

How Flat Map works

- Is the combination of a map and a flat operation
- This means you first apply the map function and then flattens the result.
- The key difference is the function used by map operation returns a Stream of values or a list of values rather than a single value, that's why we need flattening. When you flat a Stream of Stream, it gets converted into Stream of values.
- To understand what flattening a stream consists in, consider a structure like [[1,2,3],[4,5,6],[7,8,9]] which has "two levels". It's basically a big List containing three more List. Flattening this means transforming it in a "one level" structure e.g. [1,2,3,4,5,6,7,8,9] i.e. just one list.
- In short,
- Before flattening - Stream of List of Integer
- After flattening - Stream of Integer

Type here to search

Wor... *ne... Won... java... Micr... Aud... 25°C 5:54 PM ENG US 11/27/2021

java_27_2_21.pptx - Google Slides

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g103a5b83786_0_0

How Flat Map works

- emp -> emp.getCitiesWorkedIn() is a mapper function which is producing multiple values for each single input. i.e there are multiple cities for each single employees.
- [[Pune, Bangalore, Nagpur, Noida], [Pune, Bangalore, Noida]]
- flatMap() is flattening those multiple values into a single stream and remove duplicates by set.

This is the best explanation of flatMap. The values from the stream returned by the mapper are drained from the stream and are passed to the output stream. The "clumps" of values returned by each call to the mapper function are not distinguished at all in the output stream, thus the output is said to have been "flattened". Thank you! —

Type here to search

Wor... *ne... Won... java... Micr... Aud... 25°C 5:56 PM 11/27/2021

java_27_2_21.pptx - Google Slides

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g103a5b83786_0_0

Difference between map & flat map

MAP	FLAT MAP
The function you pass to the map() operation returns a single value.	The function you pass to flatMap() operation returns a Stream of value.
	flatMap() is a combination of map and flat operation.
map() is used for transformation only	but flatMap() is used for both transformation and flattening.

Type here to search

Wor... *ne... Won... java... Micr... Aud... 25°C 5:56 PM 11/27/2021

➤ WHAT IS OPTIONAL CLASS ?

The image shows a Google Slides presentation with two slides. The top slide is titled "Why Optional Objects" in red. It contains three bullet points: 1. Java Optional class provides a way to deal with null values. It is used to represent a value is present or not. Java 8 added a new class Optional available in java.util package. 2. A NullPointerException is a common issue in java applications. To prevent this, we normally add frequent NULL checks in our code to check if a variable is not empty before we use it in our program. Optional provides a better approach to handle such situations. 3. You can view Optional as a single-value container that either contains a value or doesn't (it is then said to be "empty"). The bottom slide is titled "Returning Value With get()" in red. It contains two bullet points: 1. get() can only return a value if the wrapped object is not null; otherwise, it throws a no such element exception. 2. This is the major flaw of the get() method. Ideally, Optional should help us avoid such unforeseen exceptions. Therefore, this approach works against the objectives of Optional and will probably be deprecated in a future release. The presentation is viewed in a browser window with the URL docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g111630a85b0_0_79. The Windows taskbar at the bottom shows the time as 2:24 PM on 2/17/2022.

Why Optional Objects

- Java Optional class provides a way to deal with null values. It is used to represent a value is present or not. Java 8 added a new class Optional available in java.util package.
- A NullPointerException is a common issue in java applications. To prevent this, we normally add frequent NULL checks in our code to check if a variable is not empty before we use it in our program. Optional provides a better approach to handle such situations.
- You can view Optional as a single-value container that either contains a value or doesn't (it is then said to be "empty"),

Returning Value With get()

- get() can only return a value if the wrapped object is not null; otherwise, it throws a no such element exception
- This is the major flaw of the get() method. Ideally, Optional should help us avoid such unforeseen exceptions. Therefore, this approach works against the objectives of Optional and will probably be deprecated in a future release.

Checking Value Presence

- When we have an Optional object returned from a method or created by us, we can check if there is a value in it or not with the `isPresent()` method
- This method returns true if the wrapped value is not null.
- as of Java 11, we can do the opposite with the `isEmpty` method

```
File Edit Source Refactor Navigate Search Project SQL Editor Run Database Window Help
Demo - SpringRestAPIDemo/src/main/java/com/codecdemo/SpringRestAPIDemo/Controller/RestControllerClass.java - Spring Tool Suite 4
RestControllerClass.java SpringRestAPIDemo/pom.xml Stream.class employee Optional.class
35 String[] words = new String[10];
36 Optional<String> checkNull =
37     Optional.ofNullable(words[5]);
38
39 return new ResponseEntity<>("hello", HttpStatus.OK);
40 }
41
42 @GetMapping(path = "/id/{id}")
43 public ResponseEntity<> getById(@PathVariable Integer id) {
44     Optional<Employee> e = repo.findById(id);
45     if (!e.isEmpty()) {
46         Optional<String> name = Optional.of(e.get().getName());
47         if (name.isPresent())
48             return new ResponseEntity<>(name.get().toUpperCase(), HttpStatus.OK);
49         else
50             return new ResponseEntity<>(" Name is null ", HttpStatus.NOT_FOUND);
51     } else
52         return new ResponseEntity<>(" Sorry Employee with given ID is not present", HttpStatus.NOT_FOUND);
53 }
54
55 @GetMapping(path = "/findAll")
56 public ResponseEntity<> getAll() {
57     //Boolean itContains = repo.findAll().stream().anyMatch(emp -> emp.getName().equalsIgnoreCase("new"));
58     Optional<Employee> e = repo.findAll().stream().filter(emp -> emp.getName().equalsIgnoreCase("new")).findAny();
59 }
60
```

The screenshot shows a Google Slides presentation with the title "Creating Optional Objects" in red. The slide contains a bulleted list of points about the Optional class in Java. The presentation is viewed in a browser window with the address bar showing a Google Docs link. The Windows taskbar at the bottom shows the time as 2:39 PM on 2/17/2022.

Creating Optional Objects

- We can create optional object with Optional's static method - Of()
- the argument passed to the of() method can't be null. Otherwise, we'll get a NullPointerException.
- But in case we expect some null values, we can use the ofNullable() method:
- By doing this, if we pass in a null reference, it doesn't throw an exception but rather returns an empty Optional object:

➤ FROM DATABASE FETCH ALL THE UNIQUE RECORDS ?

The screenshot shows an IDE window with a Java file named RestControllerClass.java. The code implements a REST API for an employee repository. The getAll() method uses a stream to fetch all employees, map their names to a list, and then use a HashSet to filter out duplicates. The code is as follows:

```
59
60 private String callMe() {
61     System.out.println("I am called");
62     //DB call 1 // DB call 2
63     return "Anonymous";
64 }
65
66
67 @GetMapping(path = "/findAll")
68 public ResponseEntity<?> getAll() {
69     List<String> namesInRepo = repo.findAll().stream().map(emp -> emp.getName()).collect(Collectors.toList());
70     Set<String> uniqueNames = new HashSet<>();
71     Set<String> duplicateNames = namesInRepo.stream().filter(name -> !uniqueNames.add(name)).collect(Collectors.toCollection(HashSet::new));
72
73     return new ResponseEntity<>(uniqueNames, HttpStatus.OK);
74 }
75
76 @PostMapping(path = "/add")
77 public ResponseEntity<Employee> addEmployee(@RequestBody Employee newEmployee) {
78
79     Employee employee = repo.save(newEmployee);
80     return new ResponseEntity<>(employee, HttpStatus.OK);
81 }
82
83 @PutMapping(path = "/update")
```

➤ GROUP THE NAMES FROM DB AND FIND NAME FROM DB WHICH GREATER THAN 1 LENGTH?

Find Duplicate Elements from list using Java 8 | Java 8 coding Interview Questions | Code Decode

```
62 //DB call 1 // DB call 2
63 return "Anonymous";
64 }
65 }
66
67 @GetMapping(path = "/findAll")
68 public ResponseEntity<> getAll() {
69     List<String> namesInRepo = repo.findAll().stream().map(emp -> emp.getName()).collect(Collectors.toList());
70     Map<String, Long> mapOfNames = namesInRepo.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
71     Map<String, Long> mapOfNames = namesInRepo.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
72     return new ResponseEntity<>(mapOfNames, HttpStatus.OK);
73 }
74
75 @PostMapping(path = "/add")
76 public ResponseEntity<Employee> addEmployee(@RequestBody Employee newEmployee) {
77     Employee Employee = repo.save(newEmployee);
78     return new ResponseEntity<>(Employee, HttpStatus.OK);
79 }
80
81 @PutMapping(path = "/update")
82 public ResponseEntity<Employee> updateEmployee(@RequestBody Employee newEmployee) {
83     Employee Employee = repo.save(newEmployee);
84     return new ResponseEntity<>(Employee, HttpStatus.OK);
85 }
86 }
```

➤ USAGE OF SUMMARY STATISTICS?

java_27_2_21.pptx - Google Slide

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g113a157a158_1_0

Java 8 Statistics Example: Get count, min, max, sum, and the average for numbers

Given Emp DB, fetch max aged employee, Min aged employee, find average age of all emps working in an organization etc.

`stream().mapToInt(x -> x).summaryStatistics();`

Since these statistics operations are numeric in nature, it's essential to call the `mapToInt()` method.

It provides us utility method like `getMin()`, `getMax()`, `getSum()` or `getAverage()`.

By using these general-purpose methods, you can easily do a lot of things that require a lot of code before Java 8.

672

➤ FIND SECOND AND THIRD YOUNGEST EMPLOYEE USING SLICE CONCPET?

java_27_2_21.pptx - Google Slide

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g113a157a158_1_0

How to get Slice of a Stream in Java

Input: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Output: [2, 3, 4]
Explanation: The output contains a slice of the stream from index 1 to 3.

Using skip() and limit():

Stream API in Java provides skip() method which is used to discard the other non-required elements from the stream. It also provides limit() function which is applied to fetch the new stream with the specified index as limit, in the encountered order.

```
stream.skip(startIndex) // specify the number of elements to skip
.limit(endIndex - startIndex + 1); // specify the no. of elements the stream that should be limited
// now collect it as u require
```

Demo - SpringRestAPIDemo/src/main/java/com/codecode/SpringRestAPIDemo/Controller/RestControllerClass.java - Spring Tool Suite 4

```
57     return new ResponseEntity<>(" Sorry Employee with given ID is not present", HttpStatus.NOT_FOUND);
58 }
59
60 private String callMe() {
61     System.out.println("I am called");
62     //DB call 1 // DB call 2
63     return "Anonymous";
64 }
65
66
67 @GetMapping(path = "/findAll")
68 public ResponseEntity<> getAll() {
69     List<Integer> ages = repo.findAll().stream().map(emp -> emp.getAge()).sorted().collect(Collectors.toList());
70     ages.stream().skip(1).limit(2);
71
72     return new ResponseEntity<>(ages , HttpStatus.OK);
73 }
74
75 @PostMapping(path = "/add")
76 public ResponseEntity<Employee> addEmployee(@RequestBody Employee newEmployee) {
77
78     Employee employee = repo.save(newEmployee);
79     return new ResponseEntity<>(employee, HttpStatus.OK);
80 }
81
82 @PostMapping(path = "/update")
```

- CONVERT STRING TO UPPER CASE AND JOIN THEM WITH COMMA?

```
57         return new ResponseEntity<>(" Sorry Employee with given ID is not present", HttpStatus.NOT_FOUND);
58     }
59
60     private String callMe() {
61         System.out.println("I am called");
62         //DB call 1 // DB call 2
63         return "Anonymous";
64     }
65 }
66
67 @GetMapping(path = "/findAll")
68 public ResponseEntity<> getAll() {
69     List<String> namesList = repo.findAll().stream().map(emp -> emp.getName()).collect(Collectors.toList());
70     String names = namesList.stream().map(name -> name.toUpperCase()).collect(Collectors.joining(", "));
71     return new ResponseEntity<>(names, HttpStatus.OK);
72 }
73
74 @PostMapping(path = "/add")
75 public ResponseEntity<Employee> addEmployee(@RequestBody Employee newEmployee) {
76
77     Employee employee = repo.save(newEmployee);
78     return new ResponseEntity<>(employee, HttpStatus.OK);
79 }
80
81 @PutMapping(path = "/update")
82 public ResponseEntity<Employee> updateEmployee(@RequestBody Employee newEmployee) {
```

➤ GROUPING BY ?

IT HAS THREE OVERLOADED METHOD.

java_27_2_21.pptx - Google Slides | JsonGrid - All-in-One Json Solution | +

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g11ea0650d13_0_0

Why do we need Grouping Collector?

Given a stream of objects, there are scenarios where these objects need to be grouped based on a certain distinguishing characteristic they possess.

E.G. -
Create a map with key as Age and value as list of employees in that age group.

This concept of grouping is the same as the 'group by' clause in SQL which takes an attribute, or a calculated value derived from attribute(s), to divide the retrieved records in distinct groups.

Generally what we used to do to group by is -
iterating over each object, checking which group the object being examined falls in, and then adding that object in its correct group. The group itself is held together using a Collection instance.

Type here to search

java_27_2_21.pptx - Google Slides | JsonGrid - All-in-One Json Solution | +

docs.google.com/presentation/d/1UawAii2_ATPNmLZo8mR2vrE1U4UVq_xD/edit#slide=id.g11ea0650d13_0_0

How Grouping Collector Works?

Grouping collectors use a classification function, which is an instance of the $\text{Function}\langle T, K \rangle$

T - Type of object in stream
K are the 'group names' or 'group keys'.

for every value of K there is a collection of objects all of which return that value of K when subjected to the classification function.

All these K-values and corresponding Collection of stream objects are stored by the grouping collector in a $\text{Map}\langle K, \text{Collection}\langle T \rangle \rangle$

Type here to search

grouping collector with single parameter

```
Map<Integer,List<Employee>> employeeMap  
    = employeeList.stream().collect(Collectors.groupingBy(emp -> emp.getAge()));
```

The classification function passed to groupingBy() method is the method specified as "Employee.getAge()".

As the end result of applying the grouping collector for achieving this we want a Map with keys as age and corresponding values as List of employees of that age.

2 parameter Collectors.groupingBy()- uses a user specified Collector to collect grouped elements

Whereas the 1st variant always returned a List containing the elements of a group, the 2nd variant of grouping collector provides the flexibility to specify how the grouped elements need to be collected using a second parameter which is a Collector.

So, instead of just storing the groups in resultant Map as Lists, we can instead store them in say Sets,

```
(Collectors.groupingBy(Employee.getAger(), Collectors.toSet()));
```

➤ DIFFERENCE BETWEEN ORELSE AND ORELSEGET?

The screenshot shows a Google Slides presentation with the title "Difference Between orElse and orElseGet()". The slide contains the following text:

Notice that when using `orElseGet()` to retrieve the wrapped value, the `callMe()` method is not even invoked since the contained value is present.

However, when using `orElse()`, whether the wrapped value is present or not, the default object is created. So in this case, we have just created one redundant object that is never used.

In case of `orElse()` method, still the function is called and a default object is being created, which will never be used as the value is present for the variable wrapped in the Optional object. But `orElseGet()` works just fine, and the functional interface is never even invoked.

In this simple example, there is no significant cost to creating a default object, as the JVM knows how to deal with such. However, when a method such as `callMe()` has to make a web service call or even query a database, the cost becomes very obvious.

The second screenshot shows a Google Slides presentation with the title "Exceptions With orElseThrow()". The slide contains the following text:

Given an employee table fetch the employee object with Id and return The name in upper case, but if name is not present throw an exception to stop program execution using java 8

The `orElseThrow()` method follows from `orElse()` and `orElseGet()` and adds a new approach for handling an absent value.

Instead of returning a default value when the wrapped value is not present, it throws an exception:

```
String name = Optional.ofNullable(e.get().getName()).orElseThrow() -> new  
IllegalArgumentException("The id sent has no name");
```

Collectors class :

collectors is a utility class , provides a set of methods to create common collectors.

Ex - some methods

```

package org.example.JAVA8_5.STREAM.COLLECTORS;

import java.util.*;
import java.util.stream.Collectors;

public class CollectorsDemo_01 {
    /*
    COLLECTORS :
    collectors is a utility class , provides a set of methods to create common
    collectors
    */
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Alice", "Bob", "Charlie", "Afan");
        // toList ()
        List<String> names = list.stream().filter(name ->
name.startsWith("A")).collect(Collectors.toList());
        names.forEach(s -> System.out.println(s));

        // toSet ()
        List<Integer> nums = Arrays.asList(1,2,3,4,3,5,1,5,7,8);
        Set<Integer> integers = nums.stream().collect(Collectors.toSet());
        integers.forEach(s-> System.out.println(s));

        // toCollection ()
        LinkedList<String> linkedList =
names.stream().collect(Collectors.toCollection(() -> new LinkedList<>()));
        System.out.println(linkedList);

        // joining
        String s = list.stream().collect(Collectors.joining("..", "{", "}"));
        System.out.println(s);

        String collect = list.stream().collect(Collectors.joining("_"));
        System.out.println(collect);

        // mapping and collecting
        Set<String> stringSet =
list.stream().collect(Collectors.mapping(String::toUpperCase,
Collectors.toSet()));
        System.out.println(stringSet);

        // summarizing
        IntSummaryStatistics statistics =
nums.stream().collect(Collectors.summarizingInt(num -> num));
        System.out.println("count: "+statistics.getCount()); // 10
        System.out.println("Min: "+statistics.getMin()); // 1
        System.out.println("Max: "+statistics.getMax()); // 8
        System.out.println("sum: "+statistics.getSum()); // 39
        System.out.println("average: "+statistics.getAverage()); // 3.9

        // groupingBy
        List<String> words = Arrays.asList("Hello", "world", "java", "stream",
"collecting");

        System.out.println(words.stream().collect(Collectors.groupingBy(String::length))
); // grouping 1

        System.out.println(words.stream().collect(Collectors.groupingBy(String::length,C
ollectors.joining(",")))); // grouping 2

```

```

System.out.println(words.stream().collect(Collectors.groupingBy(String::length,Collectors.counting())));

        LinkedHashMap<Integer, Long> linkedHashMap =
words.stream().collect(Collectors.groupingBy(String::length, LinkedHashMap::new,Collectors.counting())); // grouping 3
        System.out.println(linkedHashMap);

        // partitioningBy
        Map<Boolean, List<String>> partition =
words.stream().collect(Collectors.partitioningBy(ss -> ss.length() > 5));
        System.out.println(partition);

        /* EXAMPLE :
        BY MIXING ALL ABOVE METHODS.
        */

        // exp : 1 collecting names by length
        List<String> mName = Arrays.asList("Alice", "Bob", "Sahil", "Rahul",
"Charlie", "Salman");
        Map<Integer, List<String>> nameLength =
mName.stream().collect(Collectors.groupingBy(String::length));
        System.out.println(nameLength);

        // exp : 2 count word occurrences
        String sen = "hello world hello java world";
        String[] s1 = sen.split(" ");
        Map<String, Long> occ =
Arrays.stream(s1).collect(Collectors.groupingBy(w -> w, Collectors.counting()));
        System.out.println(occ);

        // exp : 3 partitioningBy Even and Odd Numbers
        List<Integer> l2 = Arrays.asList(1, 2, 3, 4, 5, 6);
        Map<Boolean, List<Integer>> partEven =
l2.stream().collect(Collectors.partitioningBy(numb -> numb % 2 == 0));
        System.out.println(partEven);

        // exp : 4 summing values in map
        Map<String,Integer> items = new HashMap<>();
        items.put("Apple",10);
        items.put("Banana",20);
        items.put("Orange",15);
        Integer sum =
items.values().stream().collect(Collectors.summingInt(value -> value));
        System.out.println(sum);

        // exp : 5 Creating a map From Stream Elements
        List<String> fruits = Arrays.asList("Apple", "Banana", "cherry");
        Map<String, Integer> fruitMap =
fruits.stream().collect(Collectors.toMap(fr -> fr, x -> x.length()));
        System.out.println(fruitMap);

        //

    }
}

```